Simulation of Engineering System 3 (ENG3036)

Assignment Report Part 1

Student Name: Fatimah Hasan **Student Number: 2888172h**

Assignment Topic: Turning Control System for a Wheeled Rover

1.0 Introduction for Part 1

This report presents the modelling simulation and validation of a turning control system for a four wheeled rover. The rover uses differential wheel speeds to change the heading and turning.

The system uses a proportional controller that compares the rover's actual yaw angle ψ with a reference heading $\psi_{\text{ref}} = 55$ and produces a voltage difference ΔV to drive the wheel motors. The mathematical model 9, 1st order ordinary differential equations describing the motor electrical dynamics, motor and wheel mechanical dynamics, and the rover's sway and yaw motion.

Mathematical Modelling & Continuous Time Simulation

1.1 State Space Model Derivation

The system is described by nine state variables forming the state vector:

- $x_1 = i_L$: current in the left motor armature (A).
- $x_2 = \omega_{ML}$: angular speed of the left motor shaft (rad/s).
- $x_3 = \omega_{WL}$: angular speed of the left wheel (rad/s).
- $x_4 = i_R$: current in the right motor armature (A).
- $x_5 = \omega_{MR}$: angular speed of the right motor shaft (rad/s).
- $x_6 = \omega_{WR}$: angular speed of the right wheel (rad/s).
- $x_7 = v$: lateral (sway) velocity of the rover (m/s).
- $x_8 = r$: yaw rate of the rover (rad/s).
- $x_9 = \psi$: yaw angle of the rover (rad).

The resulting state derivatives are:

$$\dot{x}_1 = \frac{V_L - R\,x_1 - K_E\,x_2}{L}, \tag{1}$$

$$\dot{x}_2 = \frac{B_S(x_2 - x_3) - B_S\,x_2 - K_T\,x_1}{J_M}, \tag{2}$$

$$\dot{x}_3 = \frac{B_S}{J_W}(x_2 - x_3), \tag{3}$$

$$\dot{x}_4 = \frac{V_R - R\,x_4 - K_E\,x_5}{L}, \tag{4}$$

$$\dot{x}_5 = \frac{B_S(x_5 - x_6) - B_S\,x_5 - K_T\,x_4}{J_M}, \tag{5}$$

$$\dot{x}_6 = \frac{B_S}{J_W}(x_5 - x_6), \tag{6}$$

$$\dot{x}_7 = -K_S\,x_7 + V_T\,x_8 + K_V(F_L + F_R), \tag{7}$$

$$\dot{x}_8 = -V_T\,x_8 + K_D\,x_7 + K_Y(F_L - F_R), \tag{8}$$

$$\dot{x}_9 = x_8. \tag{9}$$

Controller (Eq. 1): The proportional controller produces the voltage difference based on the heading error:

$$\Delta\psi = K_C\psi_{\text{ref}} - K_H\psi, \qquad \Delta V = G_C\,\Delta\psi$$

The motor input voltages are (from Figure 6):

$$V_{IN,L} = V_D + \Delta V, \qquad V_{IN,R} = V_D - \Delta V$$

Motor current (Eq. 2 rearranged): Isolating $\frac{di}{dt}$:

$$\frac{di_L}{dt} = \frac{1}{L}\left(V_{IN,L} - R\,i_L - K_E\,\omega_{ML}\right)$$

$$\frac{di_R}{dt} = \frac{1}{L}\left(V_{IN,R} - R\,i_R - K_E\,\omega_{MR}\right)$$

Motor shaft (Eq. 3 rearranged): where $\Delta\omega = \omega_M - \omega_W$:

$$\frac{d\omega_{ML}}{dt} = \frac{1}{J_M}\left(K_T\,i_L - B_S(\omega_{ML} - \omega_{WL})\right)$$

$$\frac{d\omega_{MR}}{dt} = \frac{1}{J_M}\left(K_T\,i_R - B_S(\omega_{MR} - \omega_{WR})\right)$$

Wheel dynamics (Eq. 4 rearranged):

$$\frac{d\omega_{WL}}{dt} = \frac{B_S}{J_W}(\omega_{ML} - \omega_{WL})$$

$$\frac{d\omega_{WR}}{dt} = \frac{B_S}{J_W}(\omega_{MR} - \omega_{WR})$$

Wheel forces (Eqs. 5, 6):

$$F_L = \frac{2K_T}{R_W}\, i_L, \qquad F_R = \frac{2K_T}{R_W}\, i_R$$

Sway velocity (Eq. 7 rearranged):

$$\frac{dv}{dt} = -K_S\, v + V_T\, r + K_V(F_L + F_R)\frac{v}{V_T}$$

Yaw rate (Eq. 8 rearranged):

$$\frac{dr}{dt} = -V_T\, r + K_D\, v + K_Y(F_L - F_R)\, R_M$$

Yaw angle:

$$\frac{d\psi}{dt} = r$$

These 9 equations form the complete state space representation $\dot{\mathbf{x}} = f(\mathbf{x}, u)$, where the input $u = \psi_{\text{ref}}$. The system is nonlinear due to the $K_V(F_L + F_R)\frac{v}{V_T}$ coupling term in Eq. 7, and the closed loop controller coupling ψ back through ΔV.

1.2 MATLAB Model Function

The 9 state equations derived in Section 1.1 were implemented in a MATLAB function `roverDerivatives.m`, which takes the current time, state vector $\mathbf{x}$, and parameter struct as inputs and returns the derivative vector $\dot{\mathbf{x}}$. The main simulation script `roverSim_Part1.m` calls this function within the RK4 loop.

```
function dxdt = roverDerivatives(t, x, p)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ROVERDERIVATIVES State derivatives for the Rover Turning Control
% System.
%
%   dxdt =roverDerivatives(t,x,p)
%
%   State vector x=[iL;wML; wWL; iR; wMR; wWR; v;r;psi]
%     x(1) iL   - Left motor current           (A)
%     x(2) wML  - Left motor angular velocity  (rad/s)
%     x(3) wWL  - Left wheel angular velocity  (rad/s)
%     x(4) iR   - Right motor current          (A)
%     x(5) wMR  - Right motor angular velocity (rad/s)
%     x(6) wWR  - Right wheel angular velocity (rad/s)
%     x(7) v    - Sway (lateral) velocity      (m/s)
%     x(8) r    - Yaw rate                     (rad/s)
%     x(9)psi   - Yaw angle                    (rad)
%
%   p is a struct containing all system parameters.
%
%   Equations from Assignment Specification:
%     Motor electrical  (eq 2):L di/dt+ Ri+KE wM =VIN
%     Motor mechanical  (eq 3):JM dwM/dt + BS(wM-wW) =KT i
%     Wheel dynamics    (eq 4):JW dwW/dt - BS(wM-wW) =0
%     Wheel forces    (eqs5,6):F = 2 KT i / RW
%     Sway dynamics     (eq 7):dv/dt + KS v -VT r =KV(FL+FR)(v/VT)
%     Yaw dynamics      (eq 8):dr/dt + VT r -KD v =KY(FL-FR) RM
%     Yaw angle:               dpsi/dt = r

% #### Unpack states #####
iL   =x(1);
wML  = x(2);
wWL  =x(3);
iR   =x(4);
wMR  =x(5);
wWR  =x(6);
v    = x(7);
r    =x(8);
psi  =x(9);

% ---- Proportional Controller (eq 1) ----
delta_psi = p.KC * p.psi_ref - p.KH * psi;   % heading error
delta_V   = p.GC * delta_psi;                % voltage difference

% ---- Motor input voltages (from Figure 6) ----
VIN_L = p.VD + delta_V;   % left motors
VIN_R = p.VD - delta_V;   % right motors

% ---- Motor-wheel speed differences ----
dw_L = wML - wWL;
dw_R = wMR - wWR;

% ###### State equations ######
% Left motor current  (eq 2, left)
diL_dt  = (VIN_L - p.R*iL - p.KE*wML) / p.L;

% Left motor speed    (eq 3, left)
dwML_dt = (p.KT*iL - p.BS*dw_L) / p.JM;

% Left wheel speed    (eq 4, left)
dwWL_dt = (p.BS*dw_L) / p.JW;

% Right motor current (eq 2, right)
diR_dt  = (VIN_R - p.R*iR - p.KE*wMR) / p.L;

% Right motor speed   (eq 3, right)
dwMR_dt = (p.KT*iR - p.BS*dw_R) / p.JM;

% Right wheel speed   (eq 4, right)
dwWR_dt = (p.BS*dw_R) / p.JW;

% ---- Wheel forces (eqs 5, 6) ----
FL = 2*p.KT*iL / p.RW;
FR = 2*p.KT*iR / p.RW;
% ---- Sway velocity (eq 7) ----
dv_dt = -p.KS*v + p.VT*r + p.KV*(FL + FR)*(v / p.VT);

% ---- Yaw rate (eq 8) ----
dr_dt = -p.VT*r + p.KD*v + p.KY*(FL - FR)*p.RM;

% ---- Yaw angle ----
dpsi_dt = r;
% ---- Pack output ----
dxdt = [diL_dt; dwML_dt; dwWL_dt; diR_dt; dwMR_dt; dwWR_dt; ...
        dv_dt; dr_dt; dpsi_dt];

end
```

Figure 1: roverDerivatives.m

```m
%% roverSim_Part1.m
%  Rover Turning Control System - Part 1
%  Standalone MATLAB simulation using RK4%
%  State vector: x = [iL; wML; wWL; iR; wMR; wWR; v; r; psi]
%  Derivative function: roverDerivatives.m

%% ==================== PARAMETERS (Appendix A) ====================
p.BS  =0.02;           % Damping coefficient      (Nm/(rad/s))
p.GC  = 11.6;           % Controller prop. gain
p.JM  = 0.003;          % Motor armature inertia    (kg m^2)
p.JW  =0.001;          % Wheel inertia             (kg m^2)
p.KC  =1.2;            % Signal conditioning gain
p.KD =18.14;          % Yaw-sway coupling gain
p.KE  =0.35;           % Back-emf constant         (V/(rad/s))
p.KH  =1.2;            % Heading compass gain
p.KS  =9.81;           % Sway damping gain
p.KT  =0.35;           % Torque constant           (Nm/A)
p.KV  =0.486;          % Sway force gain
p.KY = 29.94;          % Yaw force gain
p.L   = 0.16;          % Motor inductance          (H)
p.R   = 5.4;           % Motor resistance          (Ohm)
p.RM  = 0.124;          % Moment arm                (m)
p.RW = 0.064;          % Wheel radius              (m)
p.VD  = 3.0;            % Drive voltage             (V)
p.VT = 0.75;           % Forward velocity          (m/s)
p.psi_ref =55*pi/180;  % Reference heading         (rad)  [55 deg]

%% ==================== SIMULATION SETTINGS ====================
% Step-size justification:
%   The fastest dynamics are the motor electrical circuit with time
%   constant tau = L/R = 0.16/5.4 = 0.0296 s.  For RK4 to accurately
%   capture these dynamics we need dt << tau.  Choosing dt = 0.001 s
%   gives ~30 steps / electrical time constant, which provides good
%   accuracy with RK4 s 4th order convergence.

dt    =0.001;           % Time step (s)
T_end =10;              % Simulation duration (s)
t     =0:dt:T_end;      % Time vector
N  =length(t);        % Number of time points

%% ==================== INITIAL CONDITIONS ====================
% From Appendix A:
% psi_0 = -12 deg, r_0 = 1 rad/s, v_0 = 0 m/s
% Motor and wheel states assumed to start from rest (0)
%       iL   wML  wWL  iR   wMR  wWR  v    r    psi
x0 = [  0;   0;   0;   0;   0;   0;   0;   1;  -12*pi/180 ];


%% ==================== RK4 INTEGRATION ====================
% 4th-order RK method
% Justification: RK4 provides O(h^4) local truncation error, giving
% excellent accuracy for this stiff ish system without requiring
% implicit methods.  It is well suited for systems with moderate
% stiffness when a sufficiently small step size is used.

X = zeros(9, N);         % Preallocate state history
X(:,1) = x0;             %  initial state

for n = 1:N-1
    tn = t(n);
    xn = X(:,n);

    % Four RK4 stages
    k1 = roverDerivatives(tn,          xn,             p);
    k2 = roverDerivatives(tn + dt/2,   xn + (dt/2)*k1,  p);
    k3 = roverDerivatives(tn + dt/2,   xn + (dt/2)*k2,  p);
    k4 = roverDerivatives(tn + dt,     xn + dt*k3,      p);

    % Weighted average update
    X(:,n+1) = xn + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
end

%% ==================== EXTRACT RESULTS ====================
iL_out  =X(1,:);     % Left motor current
wML_out = X(2,:);     % Left motor speed
wWL_out =X(3,:);     % Left wheel speed
iR_out  = X(4,:);     % Right motor current
wMR_out = X(5,:);     % Right motor speed
wWR_out =X(6,:);     % Right wheel speed
v_out   = X(7,:);     % Sway velocity
r_out   = X(8,:);     % Yaw rate
psi_out =X(9,:);     % Yaw angle

% Compute control signal for reference
delta_psi = p.KC*p.psi_ref - p.KH*psi_out;
delta_V   = p.GC * delta_psi;

% Compute forces for reference
FL_out = 2*p.KT*iL_out / p.RW;
FR_out = 2*p.KT*iR_out / p.RW;


%% ==================== PLOTTING ====================
figure('Name','Rover Turning System - Part 1 (MATLAB RK4)', ...
       'Color','w', 'Position',[100 100 1000 750]);

% --- 1) Yaw angle psi(t) ---
subplot(2,2,1)
plot(t, psi_out*180/pi, 'b','LineWidth', 1.5); hold on;
plot(t, 55*ones(size(t)), 'r--', 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
ylabel('\psi (degrees)');
title('Yaw Angle \psi(t)');
legend('\psi(t)', '\psi_{ref} = 55\circ', 'Location', 'best');

% --- 2) Yaw rate r(t) ---
subplot(2,2,2)
plot(t, r_out, 'b', 'LineWidth', 1.5);
grid on;
xlabel('Time (s)');
ylabel('r (rad/s)');
title('Yaw Rate r(t)');

% --- 3) Sway velocity v(t) ---
subplot(2,2,3)
plot(t, v_out, 'b', 'LineWidth', 1.5);
grid on;
xlabel('Time (s)');
ylabel('v (m/s)');
title('Sway Velocity v(t)');

% --- 4) Motor currents ---
subplot(2,2,4)
plot(t, iL_out, 'b', 'LineWidth', 1.2); hold on;
plot(t, iR_out, 'r--', 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
ylabel('Current (A)');
title('Motor Currents');
legend('i_L(t)', 'i_R(t)', 'Location', 'best');

% --- Additional figure: Control signal ---
figure('Name','Control Signal & Forces','Color','w', ...
       'Position',[150 80 900 350]);


% --- Additional figure: Control signal ---
figure('Name','Control Signal & Forces','Color','w', ...
       'Position',[150 80 900 350]);

subplot(1,2,1)
plot(t, delta_V,'k','LineWidth',1.2);
grid on;
xlabel('Time (s)');
ylabel('\DeltaV (V)');
title('Controller Output \DeltaV(t)');

subplot(1,2,2)
plot(t, FL_out, 'b', 'LineWidth', 1.2); hold on;
plot(t, FR_out, 'r--', 'LineWidth', 1.2);
grid on;
xlabel('Time (s)');
ylabel('Force (N)');
title('Wheel Forces');
legend('F_L(t)','F_R(t)','Location','best');

%% ==================== DISPLAY KEY METRICS ====================
fprintf('\n===== Simulation Summary =====\n');
fprintf('Step size:  dt = %.4f s\n', dt);
fprintf('Duration:   T  = %.1f s\n', T_end);
fprintf('Reference:      psi_ref = 55 deg\n');
fprintf('Initial heading: psi_0   = -12 deg\n');
fprintf('Final heading:  psi_f   = %.2f deg\n', psi_out(end)*180/pi);
fprintf('Steady-state error: %.2f deg\n', 55 - psi_out(end)*180/pi);
fprintf('==============================\n');
```

Figure 2: roverSim_Part1.m

1.3 Initial Conditions, Numerical Integration Solver & Step Size Selection

Initial Conditions: The initial conditions are taken from Appendix A of the specification:

- $\psi_0 = -12$, the rover starts offset from the 55 reference, giving an initial heading error of 67.
- $r_0 = 1$ rad/s , a non zero initial yaw rate, meaning the rover is already turning at the start.

- $v_0 = 0$ m/s , no initial lateral motion.
- All motor and wheel states $(i_L, i_R, \omega_{ML}, \omega_{MR}, \omega_{WL}, \omega_{WR})$ start at zero, representing motors at rest with no prior voltage applied.

1.4 Simulation Responses and Analysis

The MATLAB RK4 simulation was run for 10 seconds. The responses are shown in Figure 3.
Yaw Angle $\psi(t)$: The yaw angle starts at (-12) and rapidly overshoots the 55 reference, reaching approximately 120 on the first peak , an overshoot of approximately 118%. The response then oscillates around the reference with a period of 1 second. The oscillations decay slowly and by $t = 10$ s the heading is still oscillating between 45 and 65, indicating a very long settling time (> 10 s).
Yaw Rate $r(t)$: The yaw rate starts at 1 rad/s and spikes to 15 rad/s as the controller applies a large initial correction. It then oscillates between positive and negative values, corresponding to the rover repeatedly overshooting and reversing direction. The amplitude decays over time, confirming the system is stable.
Sway Velocity $v(t)$: The lateral velocity oscillates in phase with the yaw motion due to the coupling terms $V_T r$ and $K_D v$ in equ(7) and (8). Peak sway reaches ± 0.5 m/s initially, decaying as the turning motion settles.
Motor Currents $i_L(t)$, $i_R(t)$: The left and right motor currents are approximately 180 *out of phase* , when the controller demands a turn, one side draws more current while the other draws less. Peak currents reach approximately ± 2.5 A.
Controller Assessment: The proportional controller with $G_C = 11.6$ produces a stable but highly underdamped response. This is **not a good design** because:

- Excessive overshoot ($\approx 118\%$)
- Very long settling time (> 10 s)
- Steady state error, since proportional only control cannot eliminate the offset between the actual and reference heading

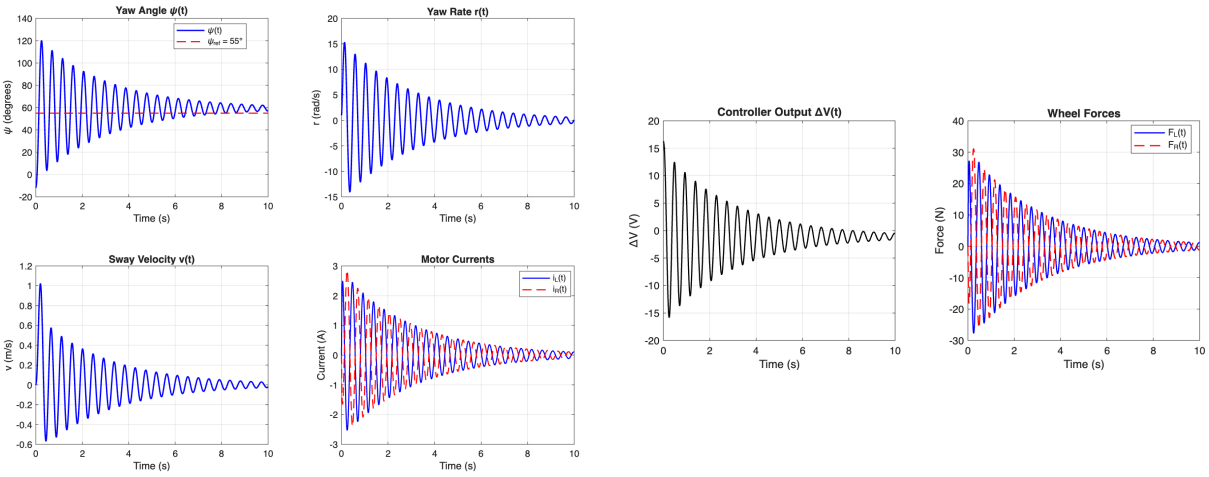


Figure 3: Simulation responses for yaw angle ψ, yaw rate r, sway velocity v, and motor currents i_L and i_R and the Controller responce.

1.5 Simulink Block Diagram

A Simulink block diagram was constructed using basic blocks to simulate the Rover Turning Control System. The model follows the structure of Figure 5, consisting of:

- **Signal Conditioning:** Gain block ($K_C = 1.2$) applied to the reference heading $\psi_{\text{ref}} = 55$.
- **Summing Junction:** Computes the heading error $\Delta\psi = K_C\psi_{\text{ref}} - K_H\psi$.
- **Turning Controller:** Gain block ($G_C = 11.6$) producing the voltage difference ΔV.
- **Heading Compass:** Feedback gain ($K_H = 1.2$) on the measured yaw angle.
- **Rover & Wheel Motors subsystem** containing:
  - Motor input voltage summing junctions ($V_{IN,L} = V_D + \Delta V$, $V_{IN,R} = V_D - \Delta V$)
  - Left and right motor subsystems, each implementing equations (2), (3) and (4) using three integrators with appropriate gain and feedback blocks
  - Force calculation using a single gain block ($2K_T/R_W = 10.9375$) for each side
  - Sway velocity subsystem implementing equation (7), including a Product block for the nonlinear $K_V(F_L + F_R)(v/V_T)$ term
  - Yaw rate subsystem implementing equation (8)
  - Yaw angle integrator ($\dot{\psi} = r$)

The solver was set to fixed step ode4 (RK) with $\Delta t = 0.001$ s and stop time of 10 s, matching the MATLAB simulation settings. The block diagram is shown below.

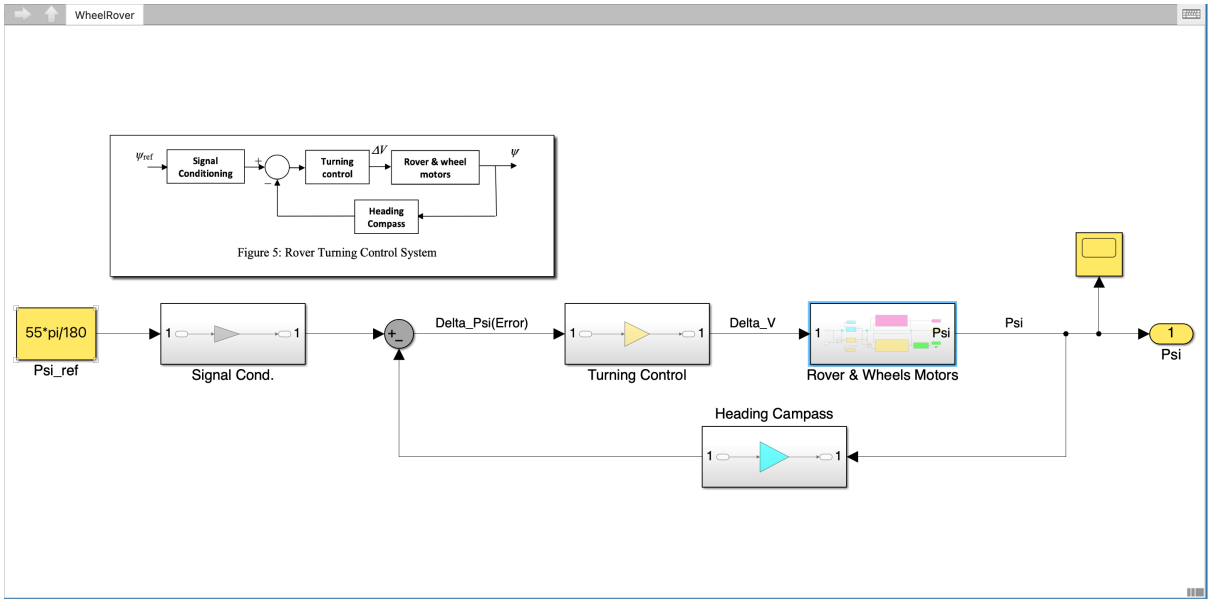


Figure 4: Wheel Rover

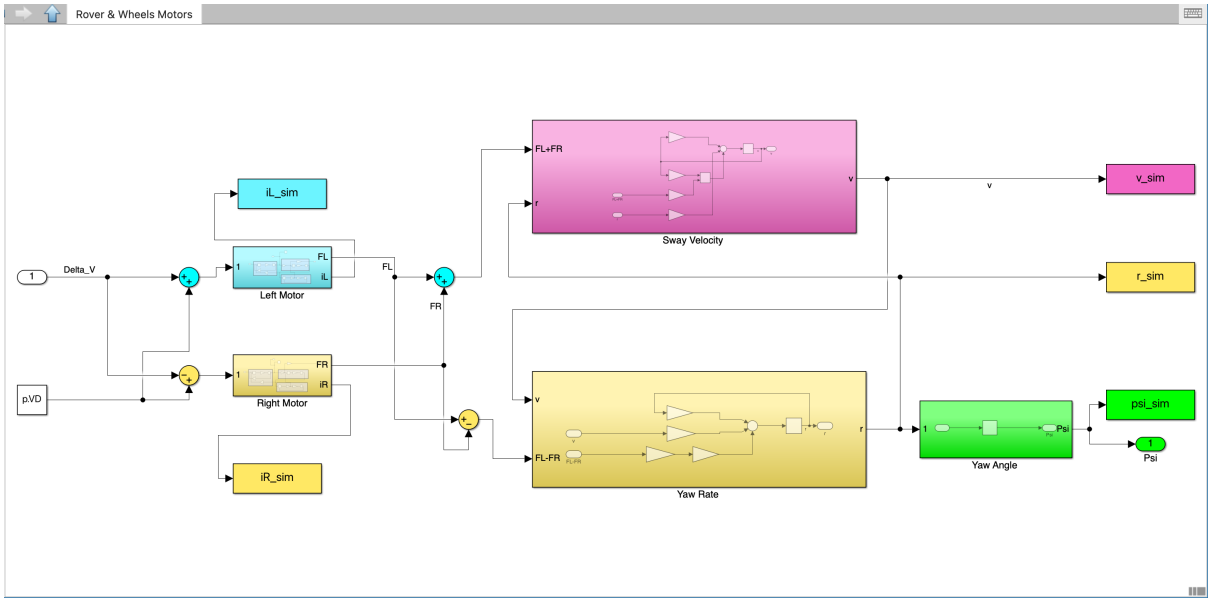


Figure 5: Wheel Rover

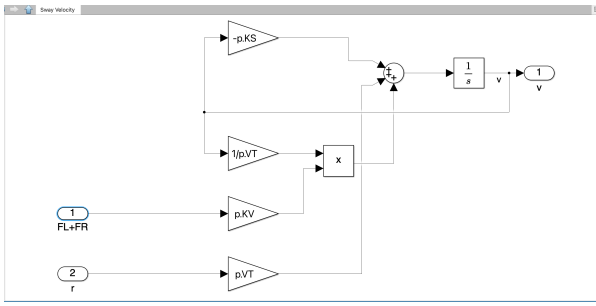


Figure 6: Sway Velocity

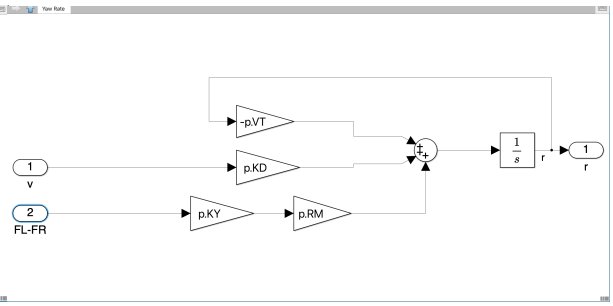


Figure 7: Yaw Rate

1.6 Validation Using Simulink Responses and Analysis

To validate the simulation, the responses from both the MATLAB script and the Simulink block diagram were overlaid on the same plots, as shown in Figure 8.

The MATLAB (blue solid) and Simulink (red dashed) responses are indistinguishable across all four outputs: yaw angle, yaw rate, sway velocity and motor current. A numerical comparison confirmed that the maximum absolute errors between the two simulations are negligible (order of 10^{-6} or smaller for all state variables).

This close agreement validates that the state space model derived in Section 1.1 and implemented in the MATLAB script correctly represents the system. Both implementations solve the same set of differential equations and produce identical dynamic responses.

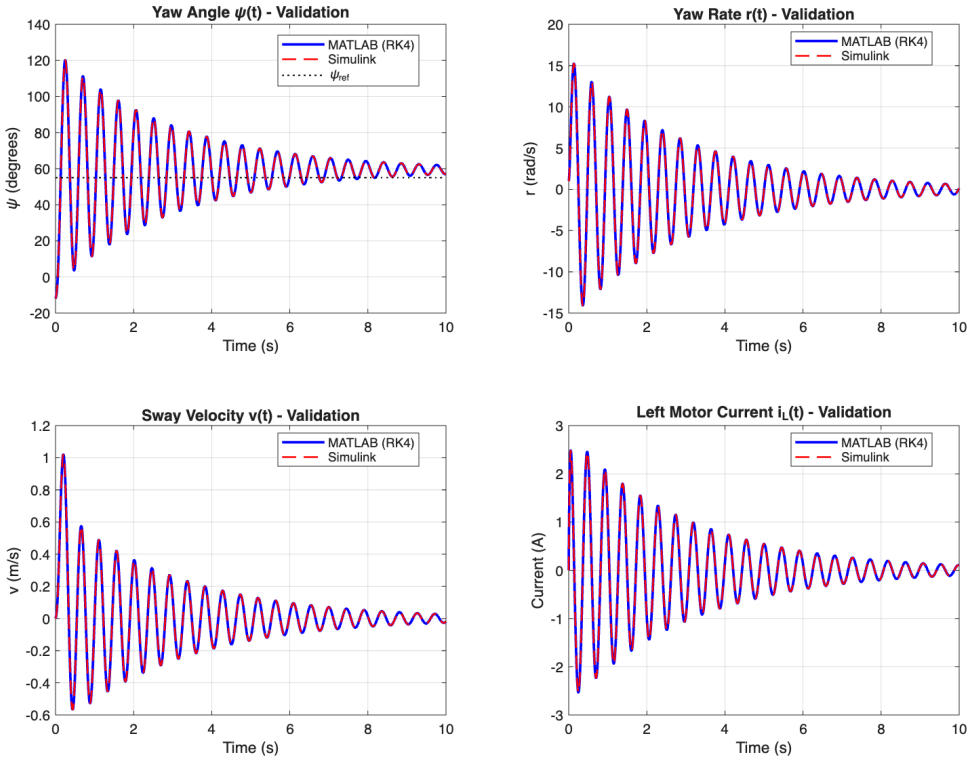


Figure 8: Yaw Angle

1.7 Conclusions for Part 1

A mathematical model of the Rover Turning Control System was successfully derived, comprising the first order ODEs describing the motor, wheel and rover dynamics under stearing control. The key findings are:

- The MATLAB and Simulink simulations produce identical results, validating the correctness of the mathematical model and its implementation.
- The proportional controller ($G_C = 11.6$) produces a stable but highly oscillatory response with approximately 118% overshoot and a settling time exceeding 10 seconds.
- A steady state heading error exists because proportional control alone cannot eliminate the offset.
- The controller is not suitable for this system in its current form and requires redesign, which is addressed in Part 2 of this report.